

Sentinelleai — Audit Report

MagnetaERC20OFT.sol · 915ed06f-89e3-46df-bffb-3330c0ed251f · 2026-05-22 11 h 46 min
13 s

FAIL 22 / 100 11 consolidated findings

MagnetaERC20OFT presents multiple severe security deficiencies across OWASP Smart Contract Top 10 2026 categories. SC01 (Access Control) is the dominant concern: a single EOA simultaneously controls all ERC-20 admin powers and LayerZero bridge configuration (setPeer, setConfig, DVN settings) without multisig, timelock, or role separation — directly paralleling the Kelp DAO and Wasabi 2026 exploits. The permissionless `autoFreeze` function (confirmed by static sentinel, attacker, historian, economist, and formalist) allows any caller to permanently blacklist any token holder above a configurable threshold using a caller-supplied amount, enabling griefing, extortion, and front-running of exits. SC03 (Logic Errors) is triggered by the critical mismatch between `enableRevokeFreeze` semantics and the unrestricted `blacklist` function — revoking freeze does not revoke manual blacklisting, creating a false immutability signal. Additional SC03 issues include `withdrawFees` sweeping the entire contract balance rather than tracked fees, and the ability to blacklist `address(0)` which bricks minting, burning, and OFT bridge flows. SC04 (Input Validation) concerns include a zero-threshold auto-freeze rule enabling mass permissionless blacklisting. The zero-address setter findings for `setTokenOpsModule` and `setMarketingWallet` are intentional design choices and are downgraded to INFO. The contract is not suitable for deployment with material value until centralization risks are mitigated with multisig governance and the permissionless blacklist write path is restricted.

Severity breakdown

1 CRITICAL 3 HIGH 3 MEDIUM 1 LOW 3 INFO

Static sentinel pre-scan

Static sentinels matched 3 pattern(s) — 1 HIGH, 2 LOW.

LOW SC01 · ZAD-1 · line 157

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setTokenOpsModule(address newModule) external onlyOwner { emit
TokenOpsModuleUpdated(tokenOpsModule, newModule); tokenOpsModule = newModule; }
```

LOW SC01 · ZAD-1 · line 221

Setter accepts zero-address (no validation)

set*(address _x) functions that write the parameter directly to state can brick the protocol if called with address(0). Add require(_x != address(0)).

```
function setMarketingWallet(address newWallet) external onlyOwner { marketingWallet =
newWallet; emit MarketingWalletUpdated(newWallet); }
```

HIGH SC01 · AST-MISSING-ACCESS-CONTROL · line 273

External state-mutating function lacks access control

An external/public, non-view function that writes to state without an onlyOwner / role / msg.sender check is callable by anyone. Even if the name doesn't match a privileged-keyword whitelist, the lack of any gate is suspicious.

```
function autoFreeze(address buyer, uint256 buyAmount) external { require(!
revokeFreezeEnabled, "MagnaERC200FT: freezing revoked"); AutoFreezeRule memory rule
= autoFreezeRule; require(rule.active, "
```

Consolidated findings

CRITICAL CVSS 9.3 SC01: Access Control

Single EOA controls all token admin and LayerZero bridge configuration without multisig or timelock

The contract owner is simultaneously the ERC-20 admin (mint, blacklist, pause, tax, metadata, revoke switches, TokenOpsModule rebinding) and the LayerZero OFT delegate (setPeer, setConfig, DVN/executor configuration). There is no multisig requirement, role separation, two-step ownership transfer, timelock, or rate limit. A single private key compromise allows an attacker to: (1) mint unlimited tokens and dump into liquidity; (2) blacklist all holders or the LP pair; (3) set tax to 25% and front-run large trades; (4) rewire LayerZero peers to attacker-controlled OApps and mint tokens cross-chain; (5) call enableRevoke* to permanently alter admin powers. This directly parallels Kelp DAO Apr 2026 (\$292M, LayerZero OFTAdapter single-verifier compromise) and Wasabi Apr 2026 (\$25M, single EOA ADMIN_ROLE with timelock=0).

constructor, onlyOwnerOrOpsModule modifier, setTokenOpsModule, mint, blacklist, setTaxFee, pause, inherited OFT/OApp delegate controls

→ Migrate owner to a multisig (e.g., Gnosis Safe 3-of-5). Separate roles for minting, blacklisting, bridge configuration, and fee management. Enforce a minimum 24-48 hour timelock for critical changes. Use two-step ownership transfer. Require DVN threshold ≥ 2 for LayerZero configuration. Consider hard-capping or irreversibly disabling mint/blacklist powers before liquidity is added.

Confirmed by: attacker, historian, formalist, developer

HIGH CVSS 8.2 SC01: Access Control

Permissionless `autoFreeze` allows any caller to permanently blacklist any token holder

The `autoFreeze(address buyer, uint256 buyAmount)` function is intentionally external and permissionless, writing to `isBlacklisted` without any access control. The caller-supplied `buyAmount` is not tied to an actual observed transfer, event, DEX buy, block, nonce, or sender. The only meaningful checks are that the rule is active, `buyAmount  $\geq$  threshold`, the buyer is not whitelisted, and `balanceOf(buyer)  $\geq$  threshold`. Any attacker can call `autoFreeze(victim, threshold)` to permanently blacklist any holder whose balance meets the threshold, preventing them from transferring, selling, burning, bridging, or receiving tokens. This enables griefing, extortion, and front-running of exits. Confirmed by static sentinel, attacker, historian, economist, and formalist agents.

function autoFreeze(address buyer, uint256 buyAmount) external – line 273

→ Restrict `autoFreeze` to a trusted relayer role or the owner/ops module. If permissionless design is retained, require a signed attestation from the off-chain listener with nonce and deadline replay protection. At minimum, add a grace period after purchase during which autoFreeze cannot be called on the buyer.

Confirmed by: static-sentinel, attacker, historian, economist, formalist, developer

HIGH CVSS 7.8 SC03: Logic Errors

`enableRevokeFreeze` does not revoke manual `blacklist` capability — false immutability signal

The contract markets `enableRevokeFreeze()` as an irreversible revocation of freezing powers, but `revokeFreezeEnabled` only blocks `pause`, `setAutoFreezeRule`, and `autoFreeze`. It does not block the `blacklist` function. After calling `enableRevokeFreeze()`, the owner and TokenOpsModule can still call `blacklist(victim, true)` to permanently freeze any holder. Users who rely on the revoke-freeze signal as a trust guarantee are misled into providing liquidity or purchasing tokens under a false immutability claim.

`enableRevokeFreeze()` and `blacklist(address account, bool value)`

→ Add `require(!revokeFreezeEnabled, 'MagnetaERC20OFT: freezing revoked')` to the `blacklist` function, or rename the flag and update documentation to clearly state that manual blacklisting remains available after freeze revocation.

Confirmed by: [attacker](#)

HIGH CVSS 7.5 SC04: Input Validation

Zero threshold in auto-freeze rule enables permissionless mass blacklisting of all holders

`setAutoFreezeRule` does not require `threshold > 0`. If `active == true` and `threshold == 0`, every address satisfies `buyAmount >= threshold` and `balanceOf(buyer) >= threshold`, including zero-balance accounts. Any attacker can then call `autoFreeze(victim, 0)` for arbitrary users, pre-blacklisting them before they ever buy or receive tokens, and can blacklist the LP pair to break all trading. This turns an admin misconfiguration into a permissionless system-wide DoS primitive.

`setAutoFreezeRule(bool active, uint256 threshold)` and `autoFreeze(address buyer, uint256 buyAmount)`

→ Require `threshold > 0` whenever `active == true`. Enforce a minimum threshold appropriate to token decimals. Prevent blacklisting known infrastructure addresses such as pools and routers.

Confirmed by: [attacker](#)

MEDIUM CVSS 6.5 SC04: Input Validation

Blacklisting `address(0)` bricks minting, burning, and OFT bridge flows

The `blacklist` function accepts any address including `address(0)`. The `\_update` hook requires both `from` and `to` to be non-blacklisted. Since ERC20 minting uses `from == address(0)` and burning/OFT sends use `to == address(0)`, blacklisting the zero address permanently breaks all mint, burn, and cross-chain bridge operations. The same issue applies to blacklisting `address(this)`, which traps collected tax fees.

```
blacklist(address account, bool value) and _update(address from, address to, uint256 value)
```

→ Reject `address(0)` and `address(this)` in the `blacklist` function. Add tests for mint, burn, and OFT flows after blacklist changes.

Confirmed by: [attacker](#)

MEDIUM CVSS 5.8 SC02: Economic / MEV

Owner can front-run large trades by instantly raising tax fee up to 25% without timelock

`setTaxFee` allows the owner to change the transfer tax up to 25% at any time with no timelock or delay. A malicious owner can monitor the mempool for large swaps and front-run them by setting the tax to maximum, extracting significant value from traders. After the trade executes, the owner resets the tax to zero. This creates a direct financial risk to traders and liquidity providers.

```
setTaxFee(uint256 newFee) — line 215
```

→ Implement a timelock for tax fee changes. The `setTaxFee` function should propose a new fee with a future activation timestamp; a separate function callable only after the timelock period applies the change.

Confirmed by: [economist](#)

MEDIUM CVSS 5.3 SC03: Logic Errors

`withdrawFees` sweeps entire contract balance rather than tracked fee accumulation

`withdrawFees` uses `balanceOf(address(this))` as the withdrawable amount with no internal `accumulatedFees` variable. Any tokens sent directly to the contract address — by users, integrators, or misconfigured contracts — are swept to the marketing wallet or owner, with no recovery path for the original sender.

```
withdrawFees()
```

→ Track taxed fees in an internal variable incremented only when tax is collected and decremented on withdrawal. Provide a separate governed rescue function for accidental transfers with timelock and clear eventing.

Confirmed by: [attacker](#)

LOW CVSS 3.1 SC04: Input Validation

Unbounded loop in `setAutoFreezeWhitelist` may exceed block gas limit

`setAutoFreezeWhitelist` iterates through an unbounded array of accounts. A very large array could exceed the block gas limit, causing a denial of service for this owner-only function. While only the owner can call this, it represents an availability risk when whitelisting a large number of accounts.

```
setAutoFreezeWhitelist(address[] calldata accounts, bool value) — line 264
```

→ Introduce a maximum batch size limit (e.g., 200 accounts per call) or implement a pagination pattern for large batches.

Confirmed by: economist

INFO CVSS 1.5 SC01: Access Control

Zero-address accepted in `setTokenOpsModule` — intentional design for module disable

The static sentinel flagged `setTokenOpsModule` for accepting `address(0)` without validation. This is an intentional design choice: setting `address(0)` disables the TokenOpsModule path, leaving the owner as the sole authorized actor. The `onlyOwnerOrOpsModule` modifier correctly handles this case. The primary risk is not the zero address itself but that the owner can rebind this privileged module to any address without timelock.

```
setTokenOpsModule(address newModule) — line 157
```

→ Document zero as a deliberate sentinel in the function NatSpec. For nonzero module updates, consider requiring a timelock and two-step accept flow from the new module address.

Confirmed by: attacker, economist, developer

INFO CVSS 1.2 SC04: Input Validation

Zero-address accepted in `setMarketingWallet` — intentional fallback to owner

The static sentinel flagged `setMarketingWallet` for accepting `address(0)` without validation. This is not directly exploitable as a protocol brick because `withdrawFees` falls back to `owner()` when `marketingWallet == address(0)`. The zero address may redirect intended marketing proceeds to the owner, but it does not lock funds.

```
setMarketingWallet(address newWallet) — line 221
```

→ Either keep zero as an explicitly documented 'fallback to owner' sentinel in the NatSpec, or provide a separate `clearMarketingWallet()` function for intentional fallback behavior to improve clarity.

Confirmed by: attacker, economist, developer

INFO CVSS 1.0 SC01: Access Control

Formalist finding on lzReceive peer/endpoint checks — inherited from OFT base

The formalist agent flagged missing ``peer[srcEid] != bytes32(0)`` and ``msg.sender == endpoint`` checks on lzReceive paths. These checks are implemented in the inherited LayerZero OFT/OApp base contracts and are not the responsibility of this contract to re-implement. No custom lzReceive override is present in this contract.

Inherited OFT/OApp lzReceive path

→ Verify that the LayerZero OFT base library version in use includes these checks. No action required in this contract unless a custom lzReceive override is added.

Confirmed by: formalist

Historical precedents

Retrieved from the local bug-bounty corpus (~20 700 findings) by vulnerability-class match.

CRITICAL `access-control` ethereum 2025-05-28 **\$12 000 000**

Corkprotocol — Access Control (2025-05-28)

https://x.com/SlowMist_Team/status/1928100756156194955

HIGH `access-control` bsc 2023-03-28 **\$8 900 000**

SafeMoon Hack — Access Control (2023-03-28)

https://twitter.com/zokyo_io/status/1641014520041840640

HIGH `access-control` linea 2024-06-01 **\$6 880 000**

VeloCore — lack-of-access-control (2024-06-01)

<https://x.com/BeosinAlert/status/1797247874528645333>

MEDIUM `bm25` 2024-05-01

loop — The contract lacks of a setter to disallow a token

<https://github.com/code-423n4/2024-05-loop-findings/issues/87>

HIGH `bm25` 2024-01-01

curves — No Access-Control in Critical Setter Function

<https://github.com/code-423n4/2024-01-curves-findings/issues/1438>

GAS `bm25` 2021-11-01

nested — Add zero-address checkers

<https://github.com/code-423n4/2021-11-nested-findings/issues/108>

HIGH `bm25` 2023-05-01

maia — `UniswapV3Staker.sol` : Setter functions for pool variables are missing access control modifiers

<https://github.com/code-423n4/2023-05-maia-findings/issues/569>

MEDIUM `bm25` 2022-05-01

rubicon — `RubiconMarket.feeTo` set to zero-address can DoS `buy` function

<https://github.com/code-423n4/2022-05-rubicon-findings/issues/319>

MEDIUM `bm25` 2023-04-01

rubicon — `feeTo` set to zero-address can DoS the `rubicall` function

<https://github.com/code-423n4/2023-04-rubicon-findings/issues/213>

MEDIUM `bm25` 2024-04-01

noya — NoyaFeeReceiver's `receiver` doesn't have a setter function

<https://github.com/code-423n4/2024-04-noya-findings/issues/1457>

MEDIUM bm25 2024-04-01

noya — admin setter function missing events

<https://github.com/code-423n4/2024-04-noya-findings/issues/73>

LOW bm25 2021-11-01

nested — Missing zero-address check in setters

<https://github.com/code-423n4/2021-11-nested-findings/issues/83>

Generated by Sentinelleai on 2026-05-22T15:52:04.872Z. Audit ID: 915ed06f-89e3-46df-bffb-3330c0ed251f.

This report combines deterministic regex+AST sentinels, optional Slither/Mythril static analysis, and a 6-model adversarial LLM panel consolidated by a Judge agent.